INTERNSHIP PROJECT · TÜRKSAT AVIATION AND COMMUNICATION

Smart School Information System

Week 5 — Intent Detection, Explainability and Hardening

PREPARED BY	Bera Ergin — s29315
INSTITUTION	PJAIT — 7th Semester
THIS WEEK	Maturing the end-to-end flow and guarding against misuse

Where We're Coming From, Where We're Going

WEEK 4 — DONE

- ✓ POST /query works
- ✓ Natural language → SQL with Claude
- ✓ RuleEngine wraps every query
- ✓ But every question is treated as SQL

Even a user who types "hello" forces Claude to generate SQL — wasteful and fragile.

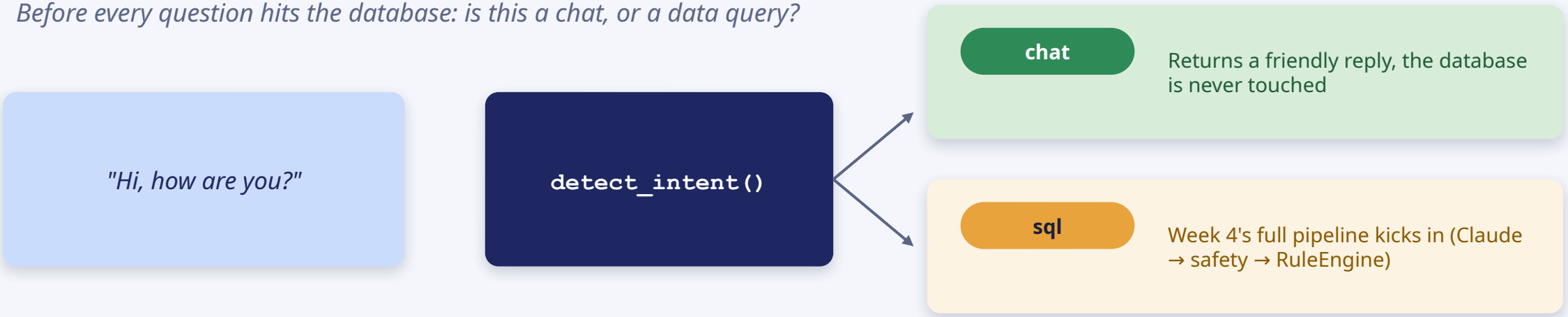
WEEK 5 — GOAL

- Intent detection — chat or query?
- A short explanation alongside the result (XAI)
- Rate limiting — closing /query to abuse
- Structured (JSON) logging

No new endpoint — the existing /query matures.

Intent Detection

Before every question hits the database: is this a chat, or a data query?



Why we designed it to be testable

detect_intent(question, client=None) — the client parameter is optional. In tests, a mock client is injected, so the 'chat' and 'sql' branches can be verified correct without making a real API call to Claude. At runtime, if no client is given, the real Anthropic client is used. This pattern is known as dependency injection, and it's what lets this service be unit-tested without an API key.

Explainability (Explainable AI)

The result isn't just a number — it explains why that result came back

Week 4 — data only

```
{
  "row_count": 4,
  "results": [...]
}
```

The user sees the number 4 but doesn't know why it's 4.

Week 5 — data + explanation

```
{
  "row_count": 4,
  "results": [...],
  "explanation": "Shown because
   you teach class 9-A —
   only that class's 4
   students are included."
}
```

The explanation text comes from the filter RuleEngine applied, not from the LLM (rule-based, deterministic) — a deliberate choice to avoid the risk of Claude hallucinating while "explaining" its own result.

Rate Limiting

Every Claude call has a cost — /query needs to be closed off to excessive use

Window	Limit	Storage	On exceeding
A 60-second sliding window	10 requests per minute per user	In-memory dict — not persistent	429 Too Many Requests + Retry-After header

A known limitation (deliberately documented)

In-memory storage means that if the server runs with more than one process/worker (e.g. multiple uvicorn workers), each worker keeps its own counter — the real limit becomes N × worker_count. This is fine for a single-worker dev/demo environment; production would need to move to a shared store like Redis. This limitation is stated explicitly in the README.

Structured Logging

Every /query request leaves behind a single-line JSON log entry

```
{
  "timestamp": "2026-10-12T14:32:01Z",
  "user": "teacher1",
  "role": "teacher",
  "intent": "sql",
  "question": "How many students are in my class?",
  "row_count": 4,
  "duration_ms": 842,
  "rate_limited": false
}
```

The raw generated SQL is logged, but since the user's question and the result may contain sensitive data, this log stays server-side only — it never leaks into the API response or the client.

No New Dependencies This Week

A deliberate choice — the standard library was enough

Rate limiting	Built with Python's built-in collections.deque and time modules — no library like slowapi was needed
Logging	Python's built-in logging module, configured with a JSON-formatted handler
Intent detection	Reuses the existing anthropic SDK (from Week 4) — no new package added

requirements.txt unchanged from Week 4: fastapi, uvicorn, python-jose, passlib, sqlalchemy, anthropic

Week 5 — Scope and Acceptance Criteria

Delivered this week

- ✓ backend/app/services/intent_service.py
- ✓ backend/app/services/explain_service.py
- ✓ backend/app/core/rate_limiter.py
- ✓ backend/app/core/logging_config.py
- ✓ Intent/explanation/limit integration into query.py
- ✓ Independent unit tests for all three services

Acceptance criteria

1. A chat intent is answered without ever touching the database
2. An SQL intent goes through Week 4's full pipeline
3. The 11th request (per minute) returns 429 with a Retry-After header
4. Every /query request produces one JSON log line
5. intent_service and rate_limiter tests pass without an API key

Next Step — Week 6

Streamlit Interface

- Login screen and query tab
- TR/EN language selection
- The first visual interface connecting to the API
- Users will finally see a real screen instead of curl

Thank you